Exp 5 :- To implement Classification algorithms (Decision tree and Naive Bayes algorithms) using Python.

Aim: To implement a Decision Tree classification algorithm using Python and a user-defined function on the preprocessed JioHotstar dataset and evaluate its performance using accuracy, precision, recall, F1-score and confusion matrix.

OBJECTIVES:

1. To preprocess and prepare the JioHotstar dataset for classification.
2. To divide the dataset into training and testing sets.
3. To build a Decision Tree classification model using the inbuilt Scikit-learn function.
4. To predict viewer drop-off using the trained model.
5. To evaluate the model using accuracy, precision, recall, F1-score and confusion matrix.
6. To implement the Decision Tree using a student-defined function.
7. To compare the results of the implementations.

THEORY:

Classification is a **supervised machine-learning technique** in which a model learns from labelled historical data and assigns a category/class to new observations.

In our JioHotstar problem, the model predicts whether a viewer will:

0 → Retained

1 → Dropped Off

Therefore, this is binary classification.

Decision Tree;

A Decision Tree is a supervised machine-learning algorithm used for classification and regression. It has three important parts:

- Root node — first decision
- Decision/internal nodes — further conditions
- Leaf nodes — final predicted class

The algorithm repeatedly splits the data based on feature values until it reaches a classification decision.

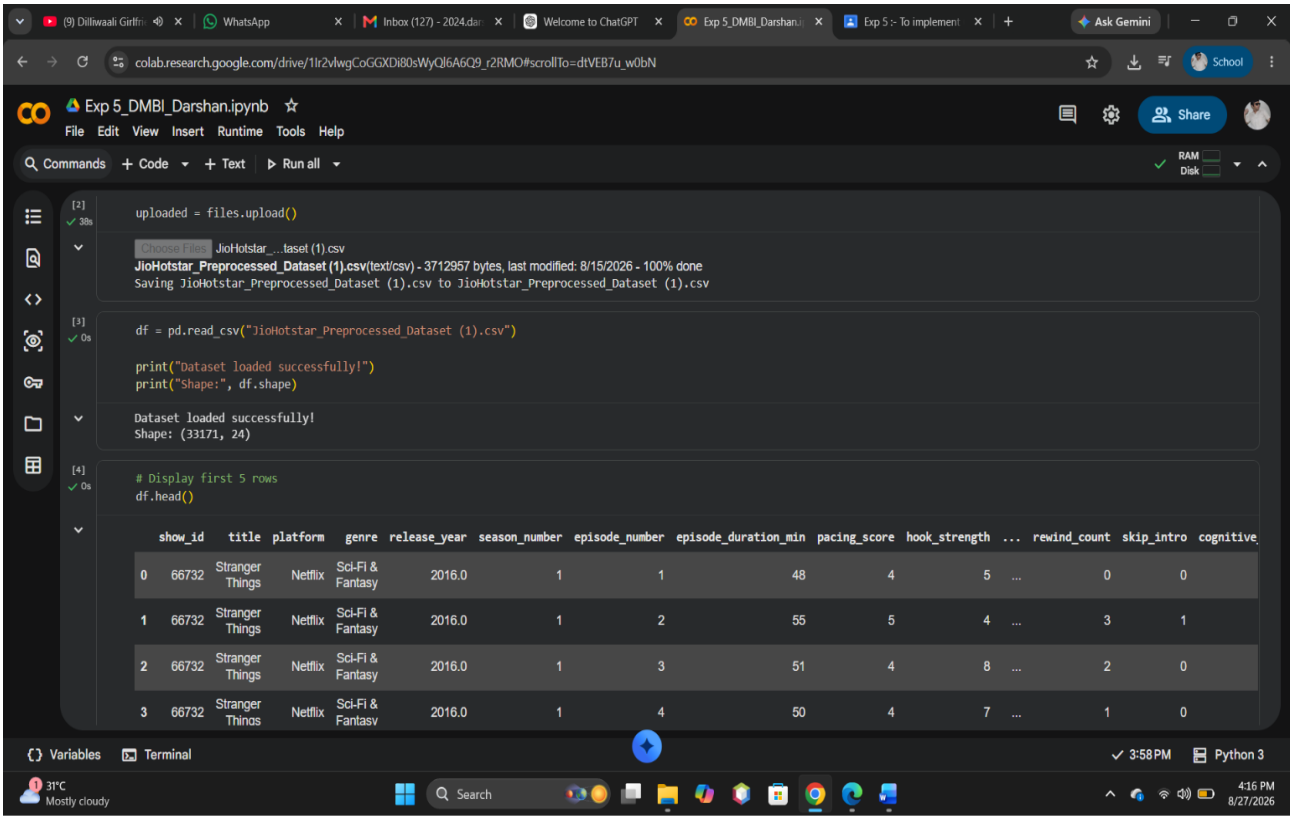
Gini Index;

criterion="gini"

The Gini Index measures the impurity of the data and helps the Decision Tree select useful splits.

Maximum Depth;

We used: max_depth=5



Loading the preprocessed JioHotstar dataset

Summary:

The preprocessed JioHotstar dataset was successfully loaded into Python using Pandas. The dataset contains 33,171 records and 24 attributes. This dataset contains information about episode characteristics and viewer behaviour and is used for classification of viewer drop-off.

```
[6]
✓ 0s
```

```python
# Check basic information about the dataset

print("Dataset Information:")
print("--------------------")

df.info()
```

```
Dataset Information:
--------------------
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 33171 entries, 0 to 33170
Data columns (total 24 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   show_id                 33171 non-null  int64
 1   title                   33171 non-null  object
 2   platform                33171 non-null  object
 3   genre                   33171 non-null  object
 4   release_year            33171 non-null  float64
 5   season_number           33171 non-null  int64
 6   episode_number          33171 non-null  int64
 7   episode_duration_min    33171 non-null  int64
 8   pacing_score            33171 non-null  int64
 9   hook_strength           33171 non-null  int64
 10  dialogue_density        33171 non-null  object
 11  visual_intensity        33171 non-null  int64
 12  avg_watch_percentage    33171 non-null  int64
 13  pause_count             33171 non-null  int64
 14  rewind_count            33171 non-null  int64
 15  skip_intro              33171 non-null  int64
 16  cognitive_load          33171 non-null  int64
 17  attention_required      33171 non-null  object
 18  night_watch_safe        33171 non-null  int64
 19  drop_off                33171 non-null  int64
 20  drop_off_probability    33171 non-null  float64
 21  retention_risk          33171 non-null  object
 22  dataset_version         33171 non-null  object
 23  watch_engagement_level  33171 non-null  object
dtypes: float64(2), int64(14), object(8)
memory usage: 6.1+ MB
```

```
0s
```

```python
# Check for missing values

print("Missing values in each column:")
print("------------------------------")

print(df.isnull().sum())
```

```
Missing values in each column:
------------------------------
show_id                   0
title                     0
platform                  0
genre                     0
release_year              0
season_number             0
episode_number            0
episode_duration_min      0
pacing_score              0
hook_strength             0
dialogue_density          0
visual_intensity          0
avg_watch_percentage      0
pause_count               0
rewind_count              0
skip_intro                0
cognitive_load            0
attention_required        0
night_watch_safe          0
drop_off                  0
drop_off_probability      0
retention_risk            0
dataset_version           0
watch_engagement_level    0
dtype: int64
```

Dataset information and missing-value analysis

Summary:

The dataset information was examined using df.info() and missing values were checked using isnull().sum(). The analysis showed that the dataset contains numerical and categorical attributes, and no missing values were present in the dataset columns. Therefore, the dataset was suitable for further classification modelling without additional missing-value treatment.

```
# Check the target variable

print("Drop-off value counts:")
print("----------------------")

print(df["drop_off"].value_counts())

print("\nDrop-off percentages:")
print("----------------------")

print(df["drop_off"].value_counts(normalize=True) * 100)
```

```
Drop-off value counts:
----------------------
drop_off
0    28367
1     4804
Name: count, dtype: int64

Drop-off percentages:
----------------------
drop_off
0    85.51747
1    14.48253
Name: proportion, dtype: float64
```

Distribution of the target variable drop_off

Summary:

The target variable drop_off contains two classes. Class 0 represents retained viewers and contains 28,367 records, while class 1 represents viewers who dropped off and contains 4,804 records. Thus, the problem is a binary classification problem. The target classes are not equally distributed, with retained viewers forming the majority class.

```
features = [
    "episode_duration_min",
    "pacing_score",
    "hook_strength",
    "visual_intensity",
    "avg_watch_percentage",
    "pause_count",
    "rewind_count",
    "skip_intro",
    "cognitive_load",
    "night_watch_safe"
]

X = df[features]
y = df["drop_off"]

print("Features selected:")
print(features)

print("\nShape of X:", X.shape)
print("Shape of y:", y.shape)
```

```
Features selected:
['episode_duration_min', 'pacing_score', 'hook_strength', 'visual_intensity', 'avg_watch_percentage', 'pause_count', 'rewind_count', 'skip_intro', 'cognitive_load', 'night_watch_safe']

Shape of X: (33171, 10)
Shape of y: (33171,)
```

```python
# Split the dataset into training and testing data

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training data:")
print("X_train:", X_train.shape)
print("y_train:", y_train.shape)

print("\nTesting data:")
print("X_test:", X_test.shape)
print("y_test:", y_test.shape)
```

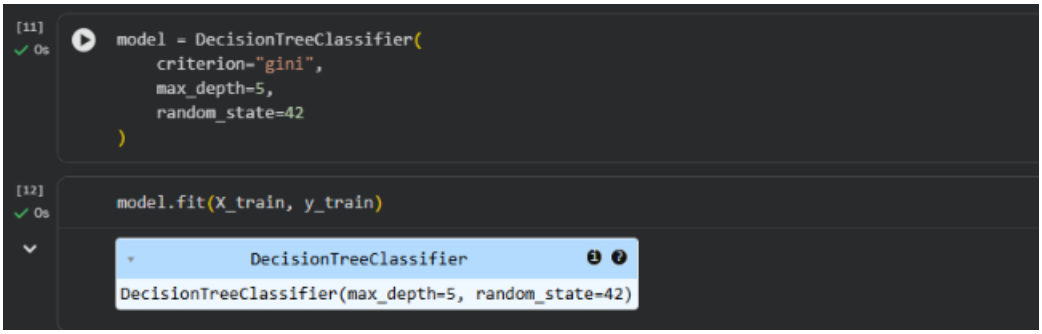
```
Training data:
X_train: (26536, 10)
y_train: (26536,)

Testing data:
X_test: (6635, 10)
y_test: (6635,)
```

Feature selection and division of dataset into training and testing data

Summary:

Ten relevant numerical and behavioural attributes were selected as input features, while drop_off was selected as the target variable. The dataset was divided into 80% training data and 20% testing data. Therefore, 26,536 records were used for training and 6,635 records were used for testing.



Training the Decision Tree classification model

Summary:

A Decision Tree classifier was created using the Gini Index as the splitting criterion and a maximum depth of 5. The model was trained using the 26,536 training records. The trained model learns relationships between the selected viewer/content features and the drop_off target.

```
[13]
✓ 0s
    # Make predictions using the trained Decision Tree

    y_pred = model.predict(X_test)

    print("Predictions completed successfully!")
    print("First 20 predictions:")
    print(y_pred[:20])

    Predictions completed successfully!
    First 20 predictions:
    [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 1 0]
```

Prediction and accuracy of the Decision Tree model

Summary:

The trained Decision Tree was applied to the testing dataset to predict whether viewers would be retained or drop off. The model achieved an accuracy of approximately 97.62%, indicating that the model correctly classified a high proportion of the testing records.

```
[15]
✓ 0s
    # Calculate precision, recall and F1-score

    print("Classification Report")
    print("---------------------")

    print(classification_report(y_test, y_pred))

    Classification Report
    ---------------------
                  precision    recall  f1-score   support

               0       0.99      0.99      0.99      5682
               1       0.91      0.92      0.92       953

        accuracy                           0.98      6635
       macro avg       0.95      0.95      0.95      6635
    weighted avg       0.98      0.98      0.98      6635
```

```
[16]
✓ 0s
    # Generate confusion matrix

    cm = confusion_matrix(y_test, y_pred)

    print("Confusion Matrix:")
    print("-----------------")
    print(cm)

    Confusion Matrix:
    -----------------
    [[5598   84]
     [  74  879]]
```

Classification report and confusion matrix

Summary:

The Decision Tree achieved high precision, recall and F1-score for both classes. For retained viewers (class 0), precision, recall and F1-score were approximately 0.99. For dropped-off viewers (class 1), precision was 0.91, recall was 0.92 and F1-score was 0.92. The confusion matrix shows 5,598 correctly classified retained records and 879 correctly classified dropped-off records, with relatively few incorrect classifications.

```
[17]
✓ 0s
# Student-defined function for Decision Tree Classification

def decision_tree_classifier(X_train, X_test, y_train, y_test):

    # Create Decision Tree model
    dt_model = DecisionTreeClassifier(
        criterion="gini",
        max_depth=5,
        random_state=42
    )

    # Train the model
    dt_model.fit(X_train, y_train)

    # Make predictions
    predictions = dt_model.predict(X_test)

    # Calculate accuracy
    accuracy = accuracy_score(y_test, predictions)

    # Display results
    print("Student-Defined Decision Tree Function")
    print("----------------------------------------")
    print("Accuracy:", accuracy)
    print("Accuracy (%):", accuracy * 100)

    print("\nClassification Report:")
    print(classification_report(y_test, predictions))

    print("Confusion Matrix:")
    print(confusion_matrix(y_test, predictions))

    return dt_model, predictions
```

Decision Tree implementation using a student-defined function

Summary:

A student-defined Python function was created to encapsulate the Decision Tree classification process. The function creates the model, trains it using the training data, generates predictions for the testing data and calculates the evaluation metrics. The function achieved the same accuracy of approximately 97.62% as the inbuilt implementation because the same training data, testing data and Decision Tree parameters were used.

```
[18]
✓ 0s
# Call the student-defined function

custom_model, custom_predictions = decision_tree_classifier(
    X_train,
    X_test,
    y_train,
    y_test
)

Student-Defined Decision Tree Function
----------------------------------------
Accuracy: 0.9761868877166541
Accuracy (%): 97.6186887716654

Classification Report:
              precision    recall  f1-score   support

           0       0.99      0.99      0.99      5682
           1       0.91      0.92      0.92       953

    accuracy                           0.98      6635
   macro avg       0.95      0.95      0.95      6635
weighted avg       0.98      0.98      0.98      6635

Confusion Matrix:
[[5598   84]
 [  74  879]]

[19]
✓ 0s
# Compare Inbuilt and Student-Defined Decision Tree

inbuilt_accuracy = accuracy_score(y_test, y_pred)
custom_accuracy = accuracy_score(y_test, custom_predictions)

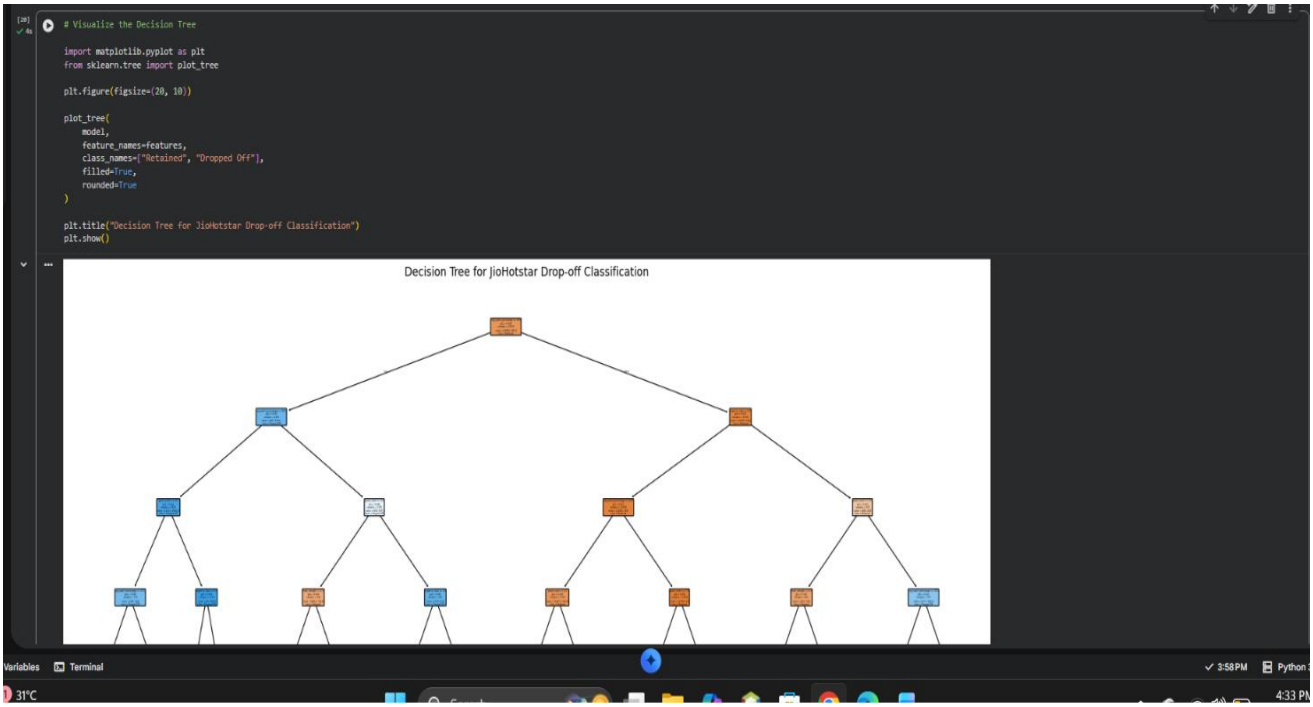
print("Comparison of Decision Tree Implementations")
print("-------------------------------------------")

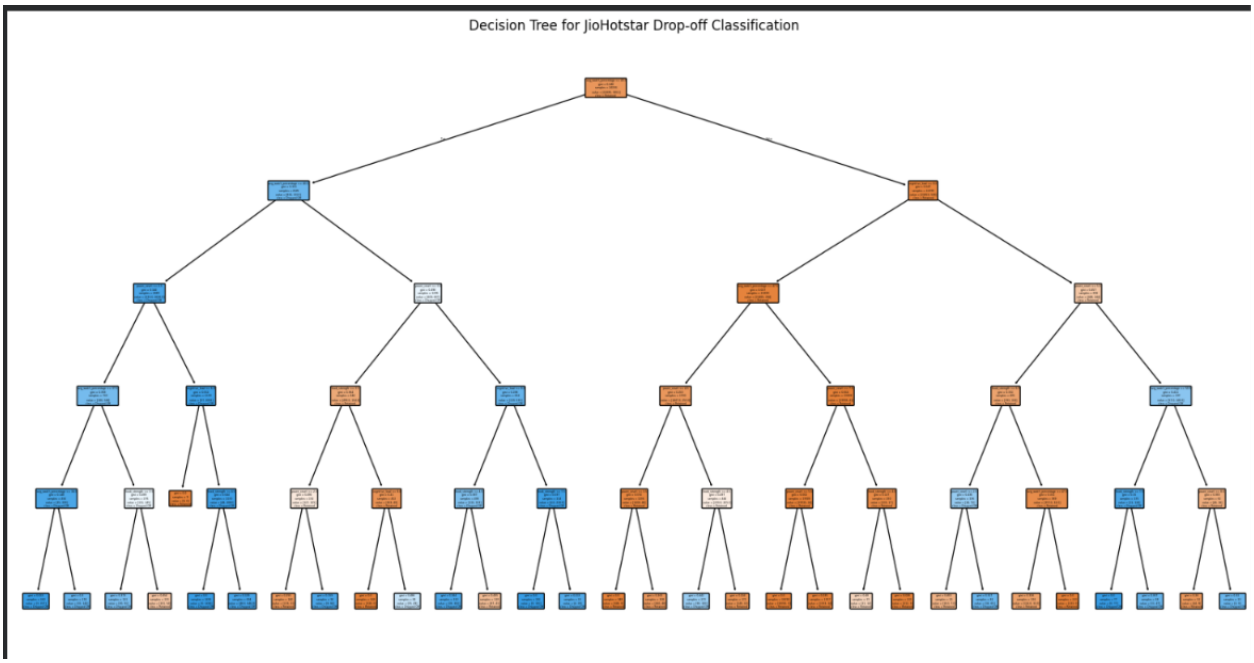
print("Inbuilt Decision Tree Accuracy      :", inbuilt_accuracy)
print("Student-Defined Function Accuracy   :", custom_accuracy)

if inbuilt_accuracy == custom_accuracy:
    print("\nBoth implementations produced the same accuracy.")
else:
    print("\nThe two implementations produced different accuracy values.")

Comparison of Decision Tree Implementations
-------------------------------------------
Inbuilt Decision Tree Accuracy      : 0.9761868877166541
Student-Defined Function Accuracy   : 0.9761868877166541

Both implementations produced the same accuracy.
```





Visualization of Decision Tree for JioHotstar drop-off classification

Summary:

The Decision Tree visualization shows the hierarchical decision-making structure of the classification model. Internal nodes represent decisions based on selected features, branches represent possible outcomes, and leaf nodes represent the final predicted classes. The visualization makes the classification process easier to interpret.

FINAL INFERENCE:

The Decision Tree classification algorithm was successfully implemented on the preprocessed JioHotstar viewing dataset using Python. The dataset was divided into 80% training and 20% testing data. The Decision Tree model using the Gini splitting criterion and maximum depth of 5 achieved an accuracy of approximately **97.62%**. The classification report showed high precision, recall and F1-score for both retained and dropped-off viewers. The confusion matrix also showed that most observations were correctly classified. A student-defined function was additionally implemented to perform the classification process. **Therefore, Decision Tree classification can effectively classify viewer drop-off behaviour using the selected JioHotstar engagement and content features.**

CONCLUSION:

Thus, the experiment successfully implemented a Decision Tree classification algorithm using Python and a student-defined function. The model provided high classification performance on the **JioHotstar** dataset, with an **accuracy** of approximately **97.62%**. The results demonstrate that viewer engagement and content-related attributes can be used to classify whether a viewer is likely to remain retained or drop off.